

Scott-Toolkit v2 — How to Use It

This is your personal cheat sheet. The toolkit handles context engineering — making sure Claude Code always knows where you left off, what you're building, and what lessons you've learned. It works alongside GSD (project management) and Superpowers (dev methodology).

Starting a Session

Just open Claude Code in your project folder. The toolkit handles the rest:

1. **Session-start hook fires automatically** — it checks for context files and shows you what's available
2. **If you see "Resume file found"** — Claude will read `.claude-resume.md` and know exactly where you left off
3. **If you see "No resume file"** — type `/scott:resume` for a guided walkthrough of getting back up to speed

If you're in your home directory (`~`), the hook shows your active projects list. Pick one and `cd` into it.

The Slash Commands

These are your workflows. Type them and Claude walks you through each one.

Command	When to use it
<code>/scott:new-project</code>	Starting something from scratch
<code>/scott:new-feature</code>	Adding a feature to an existing project
<code>/scott:resume</code>	Picking up a project after time away
<code>/scott:retro</code>	After finishing a milestone — capture lessons
<code>/scott:handoff</code>	When a prototype is ready for Gary
<code>/scott:toolkit-update</code>	When you want to improve the toolkit itself
<code>/scott:log-success</code>	Something went really well — capture it
<code>/scott:log-error</code>	Something went wrong — capture it

```
/scott:compare-sources
```

Compare context engineering sources against your toolkit

```
/scott:bypass
```

A guard hook blocked something you approved — bypass it

You don't need to memorize these. Just describe what you want to do and Claude will suggest the right one.

What Happens Automatically

When you start a session

- Hook scans for project context files (CLAUDE.md, PRD.md, todo.md, resume file)
- Hook rebuilds `~/Sites/Global/ACTIVE-PROJECTS.md` from all your project resume files
- You see a short summary of what's available

When context is about to compact

- Hook saves a mechanical backup (`.context-snapshot.md`)
- Hook tells Claude to write a proper resume file (`.claude-resume.md`)
- **You might see Claude write a file right before compaction — that's normal and intentional**

When a session ends

- Hook reminds Claude to write the resume file and update docs
- This is your safety net — even if you forget, the reminder fires

Guard hooks (always running)

- **git push blocked** — prevents accidental pushes (use `/scott:push` or confirm manually)
 - **Destructive commands blocked** — `rm -rf`, `git reset --hard`, etc. require explanation
 - **CLAUDE.md protected** — can't be overwritten without confirmation
 - **Package installs blocked** — new dependencies need your approval
-

Pausing Work (The Most Important Part)

When you're done for the day or switching projects, you have two options:

Option A: Just close Claude Code

The session-end hook will remind Claude to write the resume file. If Claude has time before the session closes, it writes `.claude-resume.md` with everything needed to continue.

Option B: Tell Claude "let's stop here"

Claude will:

1. Write/update `.claude-resume.md` with current state
2. Update CLAUDE.md's Current Status section
3. Mark completed tasks in `tasks/todo.md`

Option B is better because Claude has full context when you explicitly pause.

If context fills up mid-session

The PreCompact hook fires automatically. Claude writes the resume file before compaction compresses the conversation. After compaction, Claude reads the resume file back and continues.

The Resume File (`.claude-resume.md`)

This is the magic. It's a ~10-line file in your project root that tells the next session exactly where to pick up.

You never write this file. Claude writes it. It contains:

- What workflow/phase you were in
- What's done and what's next
- Key decisions made during the session

```
# Resume Context
Workflow: /scott:new-feature — Phase 4 (Build)
PM Mode: GSD
Branch: feat/user-auth

Done: Mini-PRD approved. Schema updated.
Current: Auth middleware written, not connected to routes.
Next: Connect middleware, then verify.
Decisions: JWT over sessions. 15min access / 7day refresh.
```

Active Projects List

The file `~/Sites/Global/ACTIVE-PROJECTS.md` is auto-generated every time you start a session. It lists all projects that have a `.claude-resume.md` file.

Don't edit this file. It gets rebuilt from resume files every session.

Multi-Machine Sync

The toolkit lives at `~/Sites/Global/scott-toolkit/` and is a git repo. To sync across machines:

```
# On any machine – pull latest
cd ~/Sites/Global/scott-toolkit && git pull

# If hooks/rules/skills seem off – re-deploy
./setup.sh
```

The setup script uses symlinks, so after `git pull` most changes take effect immediately.

Quick Reference: What Lives Where

Location	What's there	Who maintains it
<code>~/Sites/Global/scott-toolkit/</code>	The toolkit repo (source of truth)	You + Claude via <code>/scott:toolkit-update</code>
<code>~/ .claude/hooks/</code>	Deployed hooks (symlinked to toolkit)	<code>setup.sh</code>
<code>~/ .claude/rules/</code>	Behavior rules (symlinked to toolkit)	<code>setup.sh</code>
<code>~/ .claude/skills/scott-*/</code>	Deployed workflow skills	<code>setup.sh</code>
<code>~/ .claude/settings.json</code>	Hook registrations, plugins	Manual (machine-specific)
<code><project>/ .claude-resume.md</code>	Resume file for that project	Claude (automatically)
<code>~/Sites/Global/ACTIVE-PROJECTS.md</code>	Master project list	Session-start hook (automatically)

What Runs Without You Thinking About It

Everything below happens behind the scenes.

Files that write themselves

File	What it is	When	By whom
<code>.claude-resume.md</code>	Per-project handoff note (~10 lines)	Before compaction, at session end, at workflow end	Claude
<code>.context-snapshot.md</code>	Mechanical backup (git state, todos)	Right before compaction	Hook (shell script)
<code>~/Sites/Global/ACTIVE-PROJECTS.md</code>	Master list of all in-flight projects	Every session start	Hook (shell script)

Hooks that fire on their own

Hook	When	What it does	What you see
<code>session-start.sh</code>	Every time you open Claude Code	Scans for resume files, rebuilds active projects list	A 2-3 line message
<code>pre-compact.sh</code>	Context window about to compress	Saves mechanical backup, tells Claude to write resume	Claude writing a file
<code>session-end.sh</code>	Claude Code is closing	Reminds Claude to write resume + update docs	A short checklist
<code>guard-git-push.sh</code>	Claude tries <code>git push</code>	Blocks until you confirm	"Git push blocked"
<code>guard-destructive.sh</code>	Claude tries <code>rm -rf</code> , etc.	Blocks and explains why	"Blocked: [command]..."
<code>guard-claude-md.sh</code>	Claude tries to edit CLAUDE.md/MEMORY.md	Blocks until you confirm	"Modification blocked"
<code>guard-npm-install.sh</code>	Claude tries to add packages	Blocks and lists packages	"Packages blocked"

Behavior rules always loaded

Rule	What it tells Claude
claude-behavior.md	Use Superpowers for dev methodology, GSD for project management, toolkit for learning capture. Enter plan mode for complex tasks. Never mark work done without proving it works.
code-style.md	TypeScript strict mode, Vue 3 Composition API, Tailwind v4, Pinia, 2-space indent, single quotes.
n8n-sync.md	Keep local tools-needing-setup file and n8n reminder workflow in sync.

Symlinks (why updates "just work")

The hooks and rules in `~/ .claude/` are symlinks — shortcuts that point back to `~/Sites/Global/scott-toolkit/`. When you `git pull` toolkit updates, the symlinks still point to the same files, so the updates take effect instantly.

Coming Soon (Designed, Not Yet Built)

- **PM Mode switching** — Each project will declare GSD or BMAD mode in its CLAUDE.md. Workflows will automatically use the right tools and PRD template for that mode.
- **Phase auto-advancement** — Mechanical phases (create repo, generate PRD) will run without asking "ready for next phase?" Only judgment phases (approve PRD, confirm direction) will pause for your input.